ChatGPT

Executive Summary

We propose **Maya**, a personal AI assistant built as a backend-centric system to handle ~70% of routine tasks for a two-person team (Arvind=backend/AI; Lakhan=frontend). Maya will combine multiple large language models (LLMs) (e.g. OpenAI, Google Gemini, and optionally local models) with a **Retrieval-Augmented Generation (RAG)** memory system to provide accurate, context-aware responses【42†L9-L17】【40†L236-L243】. The core architecture includes: (a) an LLM *query engine* that fans out user requests to several AI models; (b) a *vector-based memory store* for long-term user knowledge and conversation history (a "digital brain")【42†L9-L17】【43†L323-L332】; (c) a *response aggregator* that ranks or fuses model outputs (inspired by Mozilla's "Star Chamber" multi-LLM consensus【35†L12-L19】); and (d) tool integrations (calendar, web search, etc.) and optional voice I/O (STT/TTS). We adopt a **RAG pipeline** (using a vector database + embeddings to fetch relevant personal data) to ground LLM answers in the user's own context【42†L9-L17】【43†L323-L332】. Prioritized MVP features include basic chat interface, multi-model response fusion, and a simple memory (user profile + chat log). By v1 (~180 days), we'll add reminders/calendar tools, richer memory/reminder management, and voice I/O. We use Python (FastAPI) on the backend, a relational+vector DB for data, and LLM APIs (OpenAI, Google Gemini). Technical choices are guided by trade-offs (e.g. OpenAI's powerful models vs. higher cost【25†L52-L60】【26†L459-L468】, open-source LLMs vs. compute requirements). Security/privacy is addressed by encrypting data in transit/at-rest and careful use of user data (e.g. anonymizing before external calls). We include a detailed **data model**, **API endpoint** design (with examples), cost estimates based on current API pricing, and a 30/90/180-day implementation roadmap with effort estimates. Where possible, we cite authoritative sources (IBM, Google Cloud, OpenAI, etc.) to support architectural decisions【40†L236-L243】【42†L9-L17】【35†L12-L19】.

Goals & Use Cases

- **Task Automation**: Maya's goal is to handle ~70% of routine tasks *automatically*. This includes answering user questions, scheduling reminders/calendar events, summarizing information, providing recommendations, and carrying out simple "daily assistant" jobs (e.g. email drafting, data lookup). (For example, IBM notes assistants can automate FAQs and process tasks via API integrations【40†L209-L217】.)
- **Conversational Interface**: Maya should engage via natural language (text/voice) with the user. It must understand context, remember personal preferences, and clarify follow-ups. Unlike a static chatbot, Maya will use *memory* to recall facts (e.g. "My wife's birthday is in June") and adapt over time【40†L236-L243】【39†L308-L312】.
- **Multi-Model Accuracy**: To improve reliability and cover diverse domains, Maya will **fan out queries** to multiple LLMs (e.g. OpenAI's GPT and Google's Gemini) and **aggregate** their responses. This ensemble approach ("multi-LLM" consensus) has been shown to yield richer, more robust answers【35†L12-L19】. The aggregator will choose or fuse the best answer based on confidence/ranking.
- **Privacy and On-Premises Potential**: User data (conversations, personal facts) is sensitive. Maya will store data securely and consider on-device or private model inference for highly sensitive content. By default, we leverage cloud APIs but design so that key processing (memory, non-sensitive inference) could run locally if needed.

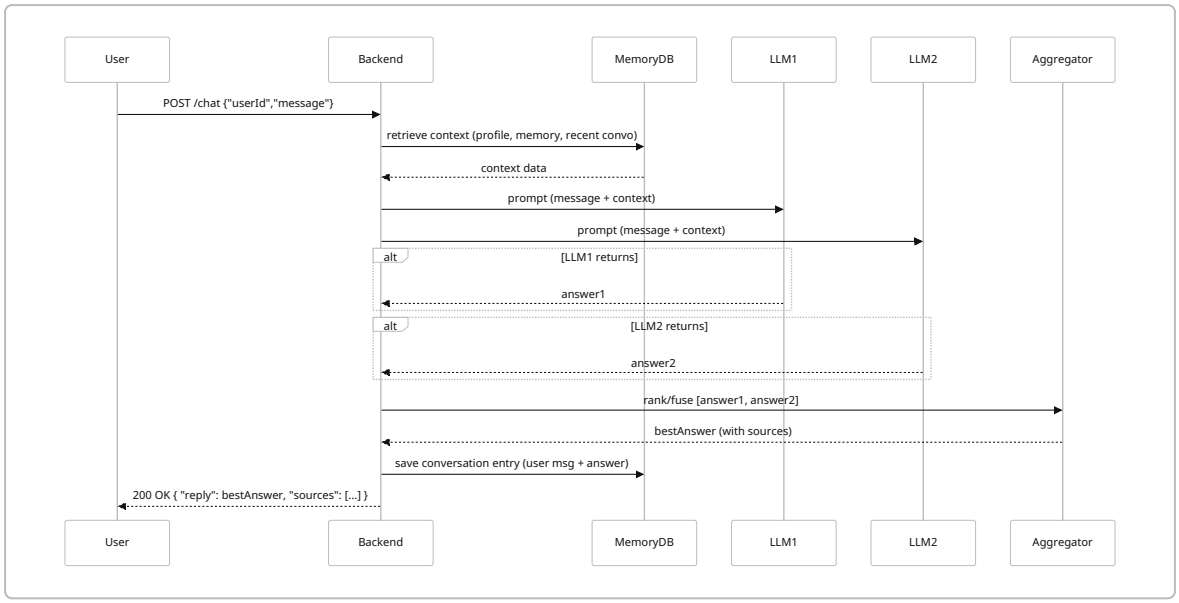
Prioritized Features (MVP→v1)

Feature / Capability	Priority	Description
Chat Interface (text)	MVP	REST API and simple UI for conversation (user queries, Maya responses).
Multi-LLM Aggregator	MVP	Route queries to two or more LLMs (e.g. GPT-5.4, Gemini-3.1), then rank/fuse their answers【35†L12-L19】.
Basic Memory (short-term)	MVP	Store recent conversation turns (contextual history) to include in prompts.
User Profile (long-term memory)	MVP	Persist user-specific data (name, preferences, bio facts) in a database to recall across sessions.
Reminders/ Scheduler	MVP	Create and track time-based reminders or calendar events.
Retrieval (RAG) System	v1 (Post-MVP)	Use a vector database (e.g. Pinecone/Weaviate) to index user documents/notes and retrieve relevant info for queries【42†L9-L17】【43†L323-L332】.
Voice I/O (STT & TTS)	v1 (Later)	Integrate speech-to-text (e.g. OpenAI Whisper【49†L129-L132】) and text-to-speech for a hands-free experience.
Tool Integrations	v1 (Later)	Integrate with external tools/APIs (web search, email, code execution) via function calls or plugins.
Security & Privacy Enhancements	Ongoing	Implement user authentication, data encryption, and compliance checks; ensure PII handling.
Analytics & Monitoring	Ongoing	Collect usage metrics, performance logs, and error monitoring (for scaling and improvement).

These priorities mirror IBM's advice that assistants excel at reacting to well-defined prompts via APIs【40†L209-L217】, and that "persistent memory" is a more advanced feature (common in agentic systems)【40†L236-L243】【39†L308-L312】. In MVP we ensure Maya can converse and recall basic info; advanced RAG and voice come later.

Architecture Overview

Maya's backend architecture is modular, combining chat handling, memory storage, retrieval, and LLM interfacing. The **sequence** of a typical request is:



1. **API Layer (FastAPI/REST)** – handles HTTP requests from the frontend.
2. **Authentication Module** – verifies user identity (e.g. JWT or Firebase Auth) and selects the correct user data.
3. **Context/Memory Manager** – queries the database for: user profile (name, preferences), past reminders/events, and **short-term memory** (recent chat history). This context is formatted into the system prompt. For larger context, we also query a **vector store** (see AI Components below) to fetch relevant personal notes or documents.
4. **LLM Engines** – we send the combined prompt to multiple LLMs (OpenAI, Gemini, etc.) asynchronously. Each returns a response (text, and possibly function-call JSON).
5. **Response Aggregator** – a small component that compares these responses. It might use a simple heuristic (e.g. longest answer, or ask a stronger model to "judge" them) or advanced ensemble techniques【35†L12-L19】. The aggregator outputs the final reply.
6. **Memory Writer** – updates the database: logs the conversation turn, any new facts (optionally fed back by the LLM), and schedules any new reminders. This ensures **persistent memory** for future sessions (unlike stateless chatbots【40†L236-L243】).
7. **Tool Executor (optional)** – if Maya identifies a tool call (e.g. calendar API, web search), the assistant emits a structured call which our system executes, then feeds the result back into the LLM. This follows agentic patterns【16†L91-L100】 but is managed within our backend.

This design follows a **RAG pipeline**: the LLM prompt is "augmented" with retrieved facts (from MemoryDB and vector search【42†L9-L17】【43†L323-L332】) to ground the response. Figure below outlines the **multi-LLM query flow and memory retrieval**.

AI Components

LLM Models

We will utilize state-of-the-art LLM APIs and possibly local models:

Model (Provider)	Context Window	Input Cost (USD per 1K tokens)	Output Cost (USD per 1K tokens)	Pros / Cons
GPT-5.5 (OpenAI)	270K tokens	$5.00	$30.00	High accuracy on complex tasks; best at reasoning【25†L37-L45】. Very expensive, slower.
GPT-5.4 (OpenAI)	270K tokens	$2.50	$15.00	Strong performance, cheaper than 5.5【25†L52-L60】. Popular baseline for assistants.
GPT-5.4 Mini (OpenAI)	270K tokens	$0.75	$4.50	Fast, low-cost; good for routine queries. Less capable on nuance.
Gemini 3.1 (Google)	~128K tokens	$1.00 (Flex) / $3.60 (Priority)	$6.00 (Flex) / $21.60 (Priority)	Strong coding/multimodal capabilities (especially Vision, TTS). Cheaper flex mode【26†L459-L468】【26†L477-L485】. Price depends on throughput needs.
Claude 3 Opus (Anthropic)	128K tokens	(est.) $1.50 / $10.00	—	[Optional] Good "assistant" tuning and privacy focus; costs ~OpenAI-tier.
Local LLaMA3 (Meta)	up to 32K tokens (3.2B model)	$0 (free model)	$0 (inference on own GPUs)	No API cost, full control, can run offline for privacy. Requires powerful GPUs; smaller context.

We expect to start with GPT-5.4 and Gemini (current strong performers【25†L52-L60】【26†L459-L468】). As a cost-saving option, GPT-5.4 Mini or a local Llama3 can handle simpler queries. See Table for detailed trade-offs. (Note: costs from OpenAI pricing【25†L52-L60】 and Gemini's published rates【26†L459-L468】; actual billing may vary.) We recommend benchmarking these models for our use cases, as model "strengths" vary by task (e.g. Gemini excels at long-context or multimodal prompts, GPT tends to be more fluent in chat).

Retrieval & Memory (RAG)

Maya uses a **Retrieval-Augmented Generation (RAG)** approach【42†L9-L17】【43†L323-L332】. All user data (notes, documents, Q&A pairs, preferences) is indexed in two places: a **relational DB** for structured info (e.g. profile fields, reminders) and a **vector store** (e.g. Pinecone, Qdrant) for semantic memories. When a query arrives, we embed it (via OpenAI or local embedding model) and perform a vector similarity search against past notes to fetch the top relevant entries. These are added to the LLM prompt as "memory context"【42†L9-L17】. This way, Maya can recall facts the user told it weeks ago, or relevant personal documents (e.g. "Resume.docx: I worked at X corp").

According to Google's RAG guidelines, grounding model input with external facts ensures accuracy and recency【42†L9-L17】【43†L323-L332】. Modern RAG systems "leverage vector databases to efficiently retrieve relevant documents"【42†L69-L77】. We will likely use an open-source vector DB (e.g. Qdrant or Weaviate) for cost-effectiveness. Entries in the vector store will include: user's important notes, FAQ-type knowledge (e.g. "what programming languages do I use?"), and structured memories (converted to text).

Ranking/Aggregator

After multiple LLMs respond, we apply a **ranking/selection step**. Possible strategies include: voting (choose the most common answer), scoring (prompting one model to judge each answer), or semantic fusion. Mozilla's "Star Chamber" code review tool fans out questions and then shows a *consensus view* 【35†L12-L19】. We can similarly, for example, use GPT-5.5 to compare answer outputs or assign confidence scores, selecting the best. Even a simple heuristic like "prefer the response from the fastest model" or "combine via concatenation" can improve robustness. Importantly, this aggregator operates in real-time within our backend (unlike user copy-pasting), providing a unified reply.

Tools & Actions

Maya should be able to **invoke tools** as needed. For example, if the prompt is "Schedule a meeting with Bob tomorrow at 3pm," the LLM could output a structured API call (using OpenAI's function-calling or a predefined schema) which our backend executes on a calendar service, then confirms back to the user. As Daegwang's agent framework shows, an LLM can emit tool calls that the system runs locally 【16†L91-L100】. We will define a toolbox of actions: calendaring, reminders, email/templates, web search (via a search API), and possibly code execution. These are advanced features (v1 timeline), but our architecture allows plugging in additional "skills" easily.

Speech (STT/TTS)

For voice interactions, we plan to use a pipeline: **Speech-to-Text → LLM Query → Text-to-Speech**. OpenAI's new real-time models (GPT-Realtime-Whisper for STT, and GPT-Realtime-TTS for voice) offer high-quality results. For example, OpenAI charges only **$0.017 per minute** for real-time Whisper transcription 【49†L129-L132】. Alternatively, we could use offline open-source STT (e.g. Whisper model on-device) for privacy, and a TTS like ElevenLabs or Google for speech output. These components (STT and TTS) will be separate microservices or library calls in the backend when voice is enabled.

Data Model

We design a simple normalized schema to store users, memory, and conversations. Below is a draft schema:

Table	Fields (Type)	Description
users	`id` (UUID), `name` (text), `email` (text), `created_at` (timestamp), `preferences` (JSON)	Registered users and their profile info.
conversations	`id`, `user_id` (FK), `user_message` (text), `assistant_reply` (text), `timestamp`	Chat history log (each turn).
memories	`id`, `user_id` (FK), `type` (text), `content` (text), `metadata` (JSON), `created_at`	Long-term memory entries (user facts, notes).

Table	Fields (Type)	Description
reminders	`id`, `user_id` (FK), `text` (text), `due_time` (datetime), `status` (enum: pending/done), `created_at`	Scheduled tasks or calendar entries.

- **Users**: Stores credentials/profile. (We may offload auth to Firebase, storing just `user_id` from Firebase in our DB.)
- **Conversations**: Logs every chat turn for context and later analysis. (This satisfies "short-term memory" by letting us reinsert recent turns into prompts.)
- **Memories**: Each row holds a user-related fact or piece of information (e.g. "Prefers skiing", or "Was at XYZ University"). The `type` could be "preference", "bio", "note", etc. These are also indexed into the vector store (embedding of `content`).
- **Reminders**: Tracks user's scheduled items. On each request, the backend can check due reminders (every minute) and proactively notify the user (push). Reminders may also be sent to the LLM if relevant.

(Note: The actual implementation might use a relational DB like PostgreSQL or Firebase for these tables, plus a separate vector DB for embeddings. The table above is illustrative – see Memory & Retrieval for the vector-indexed semantic memory.)

API Design

We expose a set of RESTful endpoints between the mobile app (Flutter) and our backend. Example endpoints:

Endpoint	Method	Request Body / Query	Response	Description
`POST /chat`	POST	`{ user_id: UUID, message: string }`	`{ reply: string, source: string[] }`	Send user query; returns Maya's reply and optional sources.
`GET /chat/history`	GET	`user_id` (query or auth)	`[{user_message, assistant_reply, timestamp}, ...]`	Fetch recent conversation turns.
`POST /memory`	POST	`{ user_id: UUID, content: string, type: string }`	`{ success: bool }`	Add a memory (e.g. user says "I love chess") to storage.
`GET /memory`	GET	`user_id` (query)	`[{id, content, type, created_at}, ...]`	Retrieve user's stored memories.

Endpoint	Method	Request Body / Query	Response	Description
`POST /reminder`	POST	`{ user_id: UUID, text: string, due_time: datetime }`	`{ success: bool, reminder_id: ID }`	Schedule a reminder.
`GET /reminder`	GET	`user_id`	`[{id, text, due_time, status}, ...]`	List reminders/ tasks for the user.
`PUT /reminder/{id}`	PUT	`{ status: \"done\" }`	`{ success: bool }`	Mark reminder as done.
`POST /voice/transcribe`	POST	Audio file/stream	`{ text: string }`	Transcribe user speech to text.
`GET /health`	GET	(no body)	`{ status: "ok" }`	Health check for monitoring.

Example Chat Request:

```
POST /chat
{
  "user_id": "123e4567-e89b-12d3-a456-426614174000",
  "message": "What's on my schedule tomorrow?"
}
```

Example Chat Response:

```
{
  "reply": "Tomorrow you have a team meeting at 10 AM and a dentist
appointment at 3 PM.",
  "source": ["MemoryReminderEntry#5", "Calendarsync"]
}
```

Here “source” might list which memory or tool provided the info. (All endpoints return JSON; errors use standard HTTP codes.)

Technology Stack Choices

We choose technologies that balance ease-of-use, cost, and performance:

- **Backend: Python + FastAPI**. FastAPI is modern, async, and well-suited for API servers. (Other options like Flask or Node could work, but FastAPI's type hints and auto-docs speed development.)
- **Database:** We start with **SQLite** or **PostgreSQL**. SQLite is easy for MVP; Postgres for production. It will store users, conversations, etc. For long-term scaling, Postgres or cloud SQL can be used.
- **Vector DB:** Options include Pinecone (managed) or Qdrant (open source). For a self-hosted preference, Qdrant or Chroma can run on our server. Pinecone offers a free tier and great support. These store embeddings for semantic memory.
- **Embeddings:** Use OpenAI's embedding API or a local model (e.g. `bge-small-en`) for generating vectors【8†L139-L144】. OpenAI's embeddings have quality and zero-maintenance (but cost); local SentenceTransformers models are free but require GPU for large sets.
- **LLM Providers:** OpenAI (GPT-5.4/5.5) and Google Gemini. We can also plug in a local LLM (Llama3, Ollama) if needed. *Pros/Cons:*
- *OpenAI GPT:* Best general performance and wide feature support (tools, function calls), but highest cost. Official docs also highlight risk of hallucination【39†L424-L432】, so our RAG context helps.
- *Gemini:* Highly capable (especially multimodal/vision) at a lower cost in flex mode【26†L459-L468】; well-integrated on Google Cloud. The Gemini Enterprise docs explicitly support RAG via Google Search and Agent Search【42†L25-L34】.
- *Local LLMs (Llama3):* Zero API cost, fully private, and can run offline. But require expensive GPUs and have limited window (<65K tokens). May use these for cached or less-critical tasks.
- **LLM Orchestration:** We use **LangChain** or **LlamaIndex** as needed. IBM mentions LangChain support for agents【39†L324-L330】; LangChain simplifies chaining queries, memory, and tool calls. LlamaIndex (GPT Index) can wrap our vector store. These frameworks accelerate development.
- **Speech (STT/TTS):** For STT, we could use OpenAI's Whisper API (GPT-Realtime-Whisper)【49†L129-L132】 or an offline model like Whisper.cpp. For TTS, services like ElevenLabs or Google Text-to-Speech have high quality; alternatively, OpenAI's GPT-Realtime-TTS (Flash) is emerging. Trade-offs: cloud APIs have a per-minute cost but superior voice quality; open-source is cheaper but may lack naturalness.
- **Frontend & Auth:** The team already chose Flutter for the mobile app. We will likely use **Firebase Authentication** (easy Flutter integration) for user login, storing only UID in our backend. This prevents us from handling passwords.
- **Hosting:** No specific cloud is mandated. For scalability, we can containerize with Docker and deploy on services like AWS/GCP/Heroku/Render. For MVP, a single VPS or cloud app instance suffices. Using Docker/ Kubernetes from day 1 is overkill for 2-person team.

Each choice has trade-offs. For instance, Google Gemini offers cheaper tokens【26†L459-L468】 but ties us to Google's ecosystem; OpenAI is "best-in-class" but more expensive【25†L52-L60】. We will likely start with APIs for speed, then consider local models once prototypes are stable. An example comparison table (excerpt):

Component	Options	Pros	Cons
LLM API	OpenAI (GPT-5.4/5.5), Google Gemini, etc.	Best quality; rich features (tools, multimodal); easy API.	Expensive per token; depends on network/ internet.
Local LLM	Llama3 (Meta), Mistral (open-source)	Free (no API cost); private (data stays local); offline use.	Requires GPU; lower context window; maintenance.
Vector DB	Pinecone (cloud), Qdrant (self-hosted)	Pinecone: SaaS, easy, scalable. Qdrant: free, local control.	Pinecone: recurring cost. Qdrant: self-maintain infra.
STT Service	OpenAI Whisper, Azure/AWS Speech, Whisper.cpp	Cloud APIs: state-of-art accuracy. Local: free.	APIs cost per minute; local may be slower/less accurate.
TTS Service	ElevenLabs, Google Cloud TTS, Coqui	High-quality, natural voice.	Can be expensive; license restrictions.
Framework	LangChain + LlamaIndex	Integrates LLMs, memory, tools seamlessly 【39†L324-L330】.	Learning curve; adds dependency.
Backend Host	Cloud VM (AWS/GCP/ Heroku) or on-prem	Cloud: scalable, managed.	Cloud: ongoing cost, vendor lock-in. On-prem: ops overhead.

(All pricing and capabilities above are indicative. Sources: OpenAI Pricing【25†L52-L60】【25†L43-L50】, Google Gemini docs【26†L459-L468】, IBM architecture guides【40†L236-L243】.)

Security & Privacy Considerations

Data security is critical since Maya will handle personal information. We will implement:

- **Authentication & Authorization:** Use secure user authentication (OAuth/JWT via Firebase Auth). Each API request must carry a token, and we verify user identity in every endpoint. We will enforce least privilege (users can only access their own data).
- **Encryption:** All communication uses HTTPS/TLS. Sensitive fields (passwords, tokens) are encrypted at rest. Note: OpenAI reports that "all data is encrypted at rest (AES-256) and in transit (TLS 1.2+)"【47†L215-L222】. We will similarly secure our own servers.
- **Data Minimization:** Only necessary user data is stored. By default, we do not send entire conversation history or PII to external models. As IBM notes, many assistants "do not inherently retain information" beyond prompts unless explicitly stored【40†L236-L243】; in Maya we *do* store memory, but we control what is sent externally. For example, we can strip out highly sensitive tokens before making an API call.
- **Privacy of Model Usage:** We will use OpenAI/Gemini APIs under the understanding that "by default, we do not use your business data for training our models"【47†L176-L183】. This means our user's queries are not fed back to the vendor for model improvement, which protects user privacy. We will also consider an enterprise contract or DPA if required (OpenAI supports GDPR compliance【47†L224-L232】).

- **Access Controls & Logging:** Implement rate limiting and IP restrictions to prevent abuse. Keep an audit log of admin actions. Regularly update dependencies and apply security patches.
- **Compliance & Safety:** Although a personal project, we will follow best practices: avoid storing credit card or health info. We will also use content filters or guardrails on LLM output to prevent it from suggesting harmful actions. As IBM warns, LLMs can hallucinate or produce unsafe suggestions【39†L424-L432】, so we may include a moderation step for high-risk queries.

Cost Estimates & Scaling Plan

Maya's costs come mainly from LLM API usage, hosting, and any paid services (TTS, vector DB). For rough estimation:

- **LLM APIs:** Suppose Maya handles 100 queries/day, each using ~1000 tokens total (prompt+response). Using GPT-5.4 ($15/M output + $2.5/M input【25†L52-L60】) we'd spend ≈$17.50 per query million tokens ≈ $1.75 per query → $175/month. Gemini flex ($6/M output + $1/M input【26†L459-L468】) would cost $0.70 per query → $70/month. Mixed usage could average ~$100–$200/month. Using the cheaper GPT-mini model for some queries could cut costs significantly (0.75/4.5 per M = ~$0.005 per 1000 tokens).
- **STT/TTS:** If we use OpenAI's Whisper at $0.017/min【49†L129-L132】, 100 minutes of speech costs $1.70. ElevenLabs TTS is ~$0.02/sec (estimate), so 1 hour is ~$72 (but maybe we keep audio short). We may defer these costs by using open-source on-device solutions or only enabling voice for Pro users.
- **Vector DB:** Pinecone's free tier supports small usage. At scale, Pinecone charges (e.g. $0.15/GB-month) – but memory data is small (a few MB). Qdrant is free if self-hosted on existing server.
- **Hosting & Database:** A small cloud VM (e.g. 2 vCPU, 4GB RAM) on AWS/GCP costs ~$20–$40/month. Database usage (Postgres, etc.) can fit in free-tier or minimal cloud SQL instance.
- **Development Tools:** We'll use open-source frameworks (FastAPI, LangChain) and free developer tiers (GitHub, VS Code).
- **Scaling:** If user load grows (say 10,000 queries/day), API cost scales linearly (~x100). At that point we'd consider caching, using GPT-mini for routine requests, or investing in a dedicated GPU server to offload some inference. We would also add load balancers and multiple backend instances. In summary, we expect **moderate monthly costs** (<$500) at MVP scale, growing to ~$1000+ if usage is high. We will monitor usage metrics and switch strategies (e.g. offloading to local models or lower-tier APIs) as needed.

Testing Strategy

To ensure reliability and quality:

- **Unit Tests:** Write automated tests for each backend component (API endpoints, database operations, memory logic). Use `pytest` for Python code. Mock external API calls (OpenAI/Gemini) during testing to avoid costs; use test keys or dummy responses.
- **Integration Tests:** Simulate end-to-end flows (user login → chat → store memory → retrieve memory). Use tools like Postman or pytest+requests. Verify data persists correctly and responses conform to expected format.
- **Regression & LLM Validation:** Because LLM outputs vary, we will create a test set of sample prompts with "golden answers" or at least checks (e.g. non-empty, correct data). We can use an LLM-based rubric (LLM-as-judge【31†L1-L4】) to score responses, catching regressions if changing models.

- **Performance Testing:** Load test the API (e.g. with Locust) to ensure it handles bursts of queries and that vector DB lookups scale. Monitor latency (we want <3 sec per chat ideally).
- **Security Testing:** Use automated scanners (e.g. OWASP ZAP) against the API. Ensure no SQL injection (use ORM with parameterized queries). Check auth flows are secure.
- **User Testing:** As features land, have the frontend testers (Lakhan or pilot users) try Maya's chat and report issues. Focus on whether answers make sense and memory works (e.g. ask a Q from prior session).
- **Continuous Monitoring:** In production, log all errors and unusual behavior. Track key metrics (API latency, cost, usage patterns) and alerts (e.g. if LLM returns error code 500, or memory DB fails). This will guide iterative fixes.

Implementation Roadmap (30/90/180 days)

We break the project into milestones with estimated effort. Team: Arvind (Backend/AI) and Lakhan (Flutter frontend) working in parallel.

Milestone	Timeline	Backend (Arvind) Tasks	Frontend (Lakhan) Tasks	Time Estimate (days/ person)
MVP (basic chat)	0–30 days	- Set up FastAPI project and database schema (users, convos, memories) - Integrate user auth (e.g. Firebase) and session management - Implement `/chat` endpoint with a single LLM (OpenAI GPT-5.4) and conversation logging - Simple memory: store and retrieve recent messages for context - Test LLM integration with sample prompts	- Build basic chat UI in Flutter: text input, display messages - Hook login auth; send user ID to backend - Test chat flow (send message → get reply) - Simple UI for profile and logout	Backend: 15, Frontend: 15
		- **Internal review:** Validate basic Q&A; ensure accurate chat formatting.		

Milestone	Timeline	Backend (Arvind) Tasks	Frontend (Lakhan) Tasks	Time Estimate (days/ person)
Enhanced MVP	30–60 days	- Add second LLM (Gemini). Develop **aggregator** logic to combine answers (start with simple “prefer GPT answer” then refine). - Implement user profile API (store name, preferences via `/memory` endpoint) - Add `/memory` endpoints to add and fetch memories. - Enhance `/chat`: include profile + recent convo + a top-k RAG retrieval (using a simple text search or small vector store) in prompt context【42†L9-L17】. - Integrate reminders: `/reminder` endpoints and a background job (cron) to send due reminders back to `/chat` or push.	- Improve UI: let user input name and preferences; add buttons for reminders view. - Display system logs (for dev) and any error messages. - Add screen to list memory entries and reminders.	Backend: 25, Frontend: 20
		- **Testing:** Unit tests for new APIs; mock LLMs to test aggregator logic; iterate on memory retrieval (use small Chroma/Qdrant for dev).		
v1 Base	60–90 days	- Refine **RAG memory**: implement real vector DB (e.g. Pinecone/ Qdrant). Index all saved memories; on each `/chat`, perform semantic search to pull relevant facts【42†L25-L33】【43†L430-L439】. - Add **ranking model**: experiment with using a “judge” LLM to pick best answer【35†L12-L19】. - Enhance `/chat`: detect and handle function calls (e.g. if LLM suggests a tool). Build one tool (e.g. calendar API call) as proof-of-concept. - Security hardening: rate limiting, input validation. Ensure HTTPS.	- Finalize chat UI polish. Add voice interface mockup (record/play UI) for next phase. - Integrate profile pic, styling, and error states.	Backend: 30, Frontend: 15

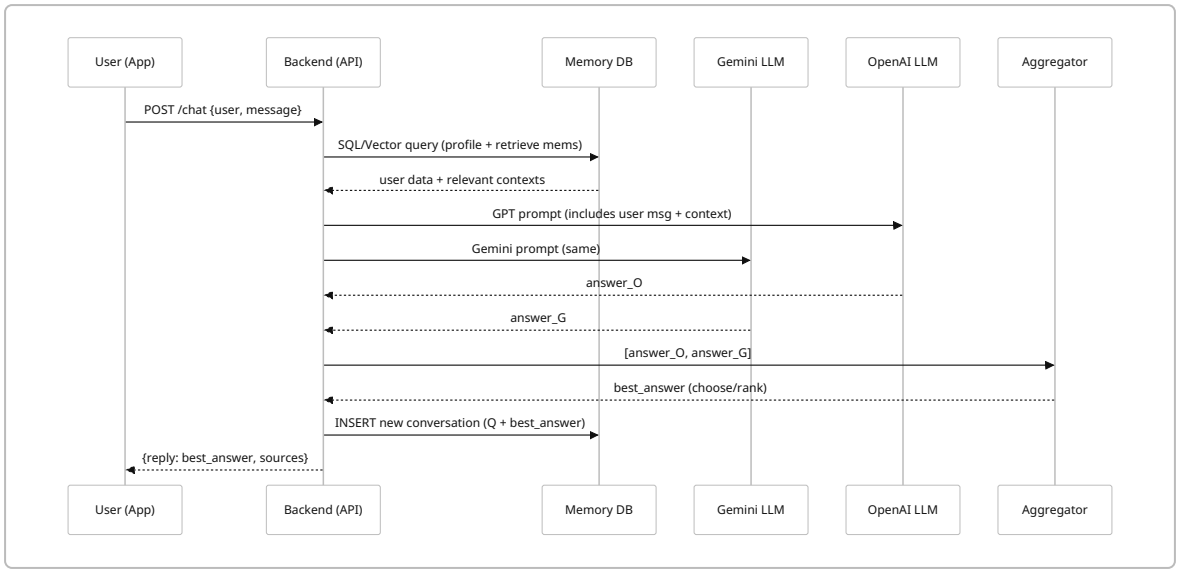
Milestone	Timeline	Backend (Arvind) Tasks	Frontend (Lakhan) Tasks	Time Estimate (days/ person)
		- **Scaling & Cost:** Evaluate token usage on ~10K tokens/week; adjust model mix for cost. Plan for autoscaling or queueing if load increases.		
v1 Plus (Voice & Tools)	90–180 days	- Implement **STT** and **TTS** integration: choose Whisper API or local and a TTS service; expose endpoints (`/voice/transcribe`). - Add more tools: e.g. web search or task automation via function calls. Use frameworks (LangChain) to simplify tool chaining. - Polish memory: allow editing/deleting memories via API. Ensure vector store refresh. - Deploy architecture on a scalable cloud environment (Docker, CI/CD).	- Add audio recording/playback UI. Integrate with backend voice endpoints. - UX improvements: persistent menu for reminders, voice button, etc.	Backend: 25, Frontend: 15
		- **Testing & Launch:** Full end-to-end tests (unit+integration) for all flows, including voice. Security audit. Prepare beta release.		

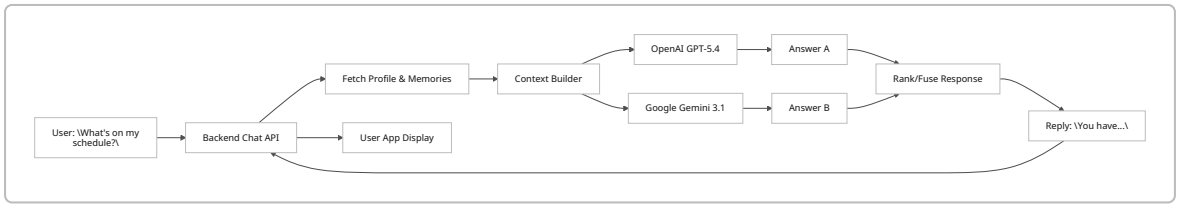
In total, Arvind's tasks are backend-heavy (authentication, LLM integration, memory, tools). Lakhan handles Flutter UI. We estimate ~90 person-days (with overlap, this is ~4 weeks of parallel work). Realistically, building a robust assistant may take longer; we budget 180 days (6 months) to reach a polished v1 with voice and tools.

Time Estimates: Each table entry approximates dedicated person-days. For instance, "Backend: 15" means ~3 weeks of Arvind's effort in that phase (assuming part-time multitasking). These include development *and* testing/debugging. Having two full-time devs could shorten calendar time.

Sequence Diagrams

Below are simplified flow diagrams. First, **User Query Flow** shows how a message is processed; second, **Multi-LLM Aggregation** illustrates how multiple model calls are combined.





These diagrams illustrate Maya's pipeline: user message → retrieve context → parallel LLM calls → aggregator → final answer → response to user (plus memory update).

References

We have drawn on numerous sources for best practices and data: IBM on AI assistants vs. agents【40†L236-L243】【39†L308-L312】, Google/Databricks on RAG【42†L9-L17】【43†L323-L332】, Mozilla on multi-LLM consensus【35†L12-L19】, and official docs for model pricing【25†L52-L60】【26†L459-L468】 and privacy【47†L176-L183】. These informed our design of Maya's memory architecture, multi-model strategy, and security posture.
